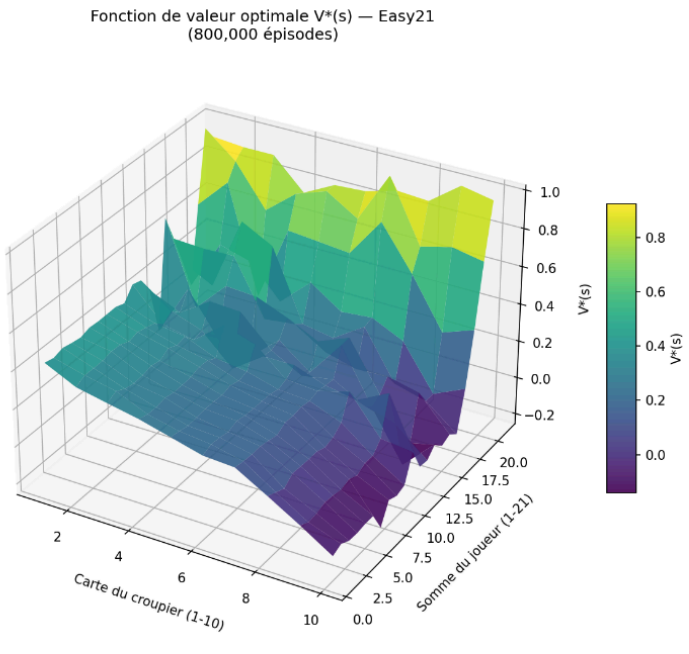
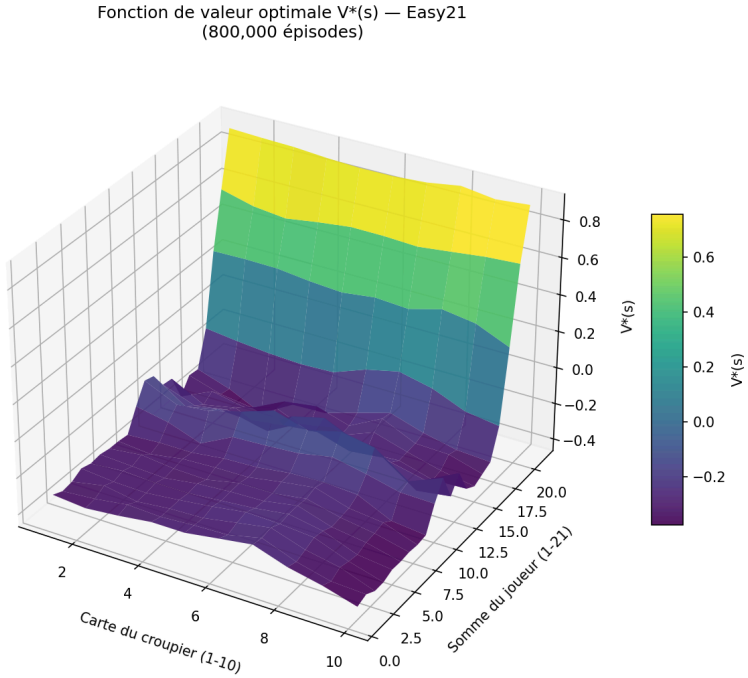
Rapport Projet Apprentissage par Renforcement :

Par Pierre-Alexandre Monceau
Enseignante : Silvia Tulli

Algorithme de Monte-Carlo Contrôle :

Voici les résultats que l'on obtient pour la fonction de valeur optimale au bout de 800 000 épisodes afin que le résultat ait bien convergé.
La différence entre ces deux graphiques provient d'un changement d'une règle dans l'environnement.
A gauche, si la somme des cartes du croupier est négative ou nulle, le joueur gagne, tandis qu'à droite, la somme des cartes du croupier peut être négative.
On remarque donc bien que la première règle avantage le joueur car les valeurs de la fonction de valeur sont bien supérieures à celles de la seconde règle.
Dans la suite du projet, nous allons choisir cette règle qui favorise le joueur (le croupier ne peut donc pas être négatif ou nul).

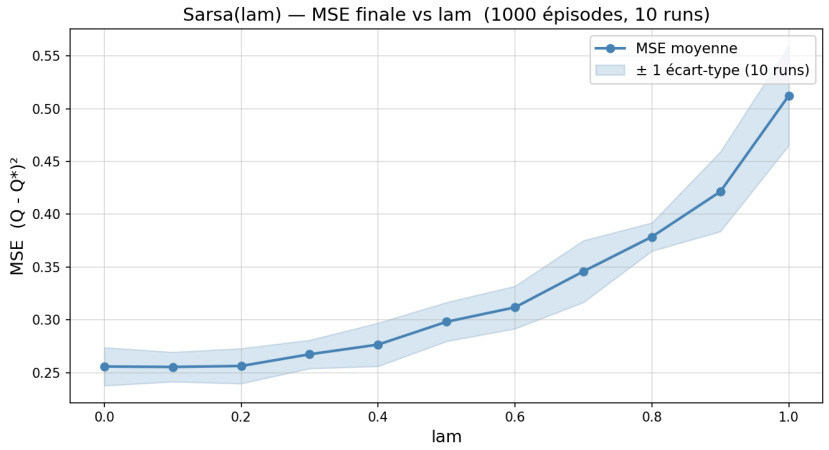




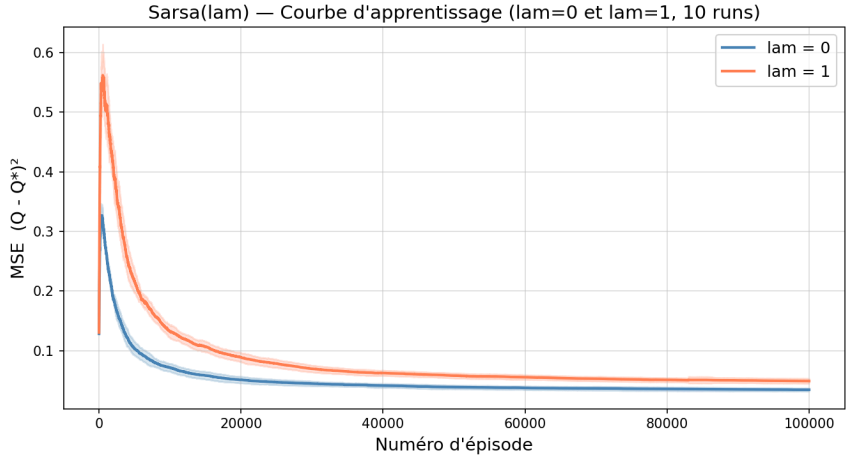
Performance en fonction de N0 :
Plus N0 est petit, plus ε va diminuer rapidement et donc plus l'algorithme va "exploiter" les résultats. Un N0 trop petit peut ainsi nous mener vers une solution correspondant à un minimum local alors que l'on cherche le minimum global. A l'inverse, un N0 trop élevé entraînera une exploration trop élevée. Cependant tous les états seront visités et l'algorithme finira par trouver le minimum global au bout d'un grand nombre d'épisodes. N0 = 100 est un bon compromis dans le cas d'Easy21 et c'est ce que nous utiliserons dans la suite du projet.

Algorithme de Sarsa(λ) :

Performance en fonction de λ :



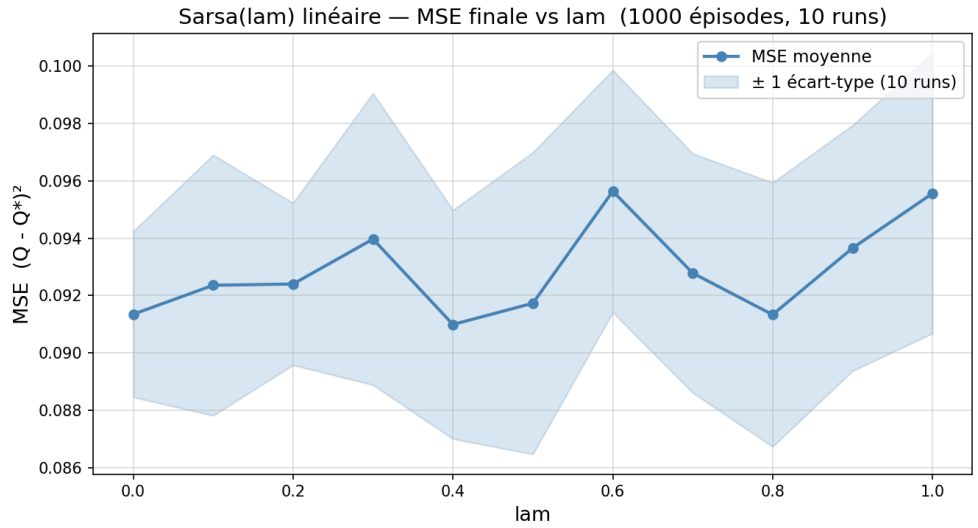
Comparaison de l'erreur quadratique moyenne entre λ=0 et λ=1 :



Ces résultats sont logiques, car les valeurs de λ produisent une famille de méthodes couvrant un spectre allant de Monte Carlo à une extrémité (λ = 1) aux méthodes TD à une étape à l'autre extrémité (λ = 0). Les performances de TD (λ = 0) sont meilleures que celles de Monte Carlo (λ = 1). Cela s'explique par le fait que la méthode MC présente une variance élevée, c'est-à-dire qu'elle trouve des estimations qui minimisent l'erreur quadratique sur le retour observé, tandis que la méthode TD présente une variance faible et trouve les estimations exactes pour le modèle de Markov. Easy21 étant un environnement de Markov, il est donc normal que Sarsa(λ = 0) performe mieux.

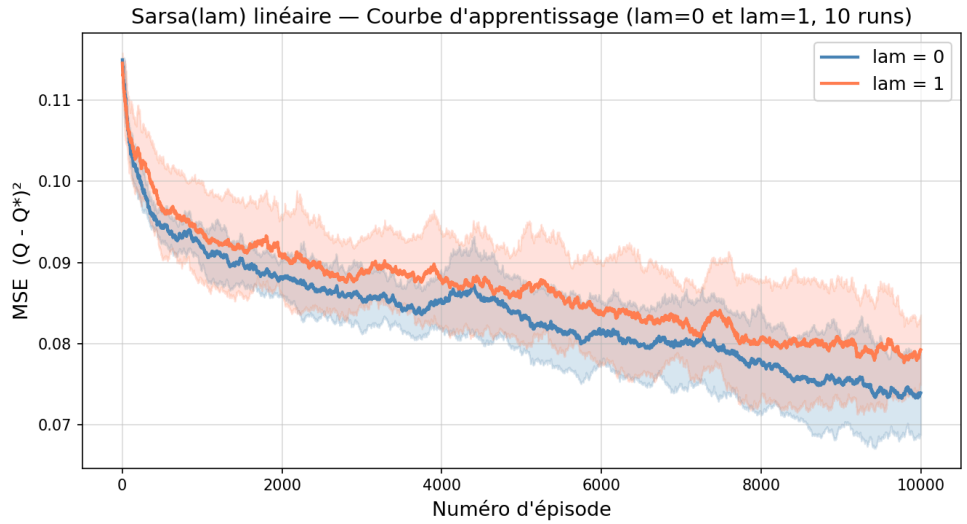
Approximation linéaire de la fonction de valeur :

Performance en fonction de λ :



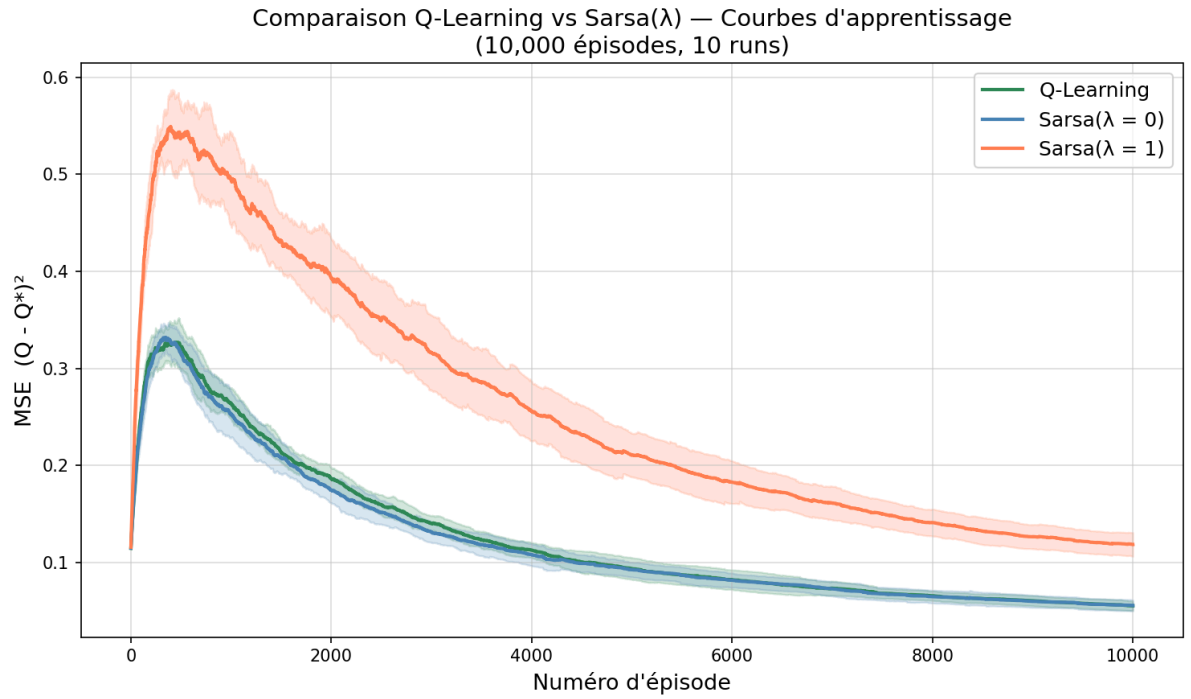
On observe que λ n'a pratiquement pas d'influence sur la performance finale de l'approximateur linéaire. Le résultat attendu : TD meilleur que MC est noyé dans le bruit introduit par le biais d'approximation. En d'autres termes, l'erreur irréductible due au codage grossier (les 36 paramètres ne peuvent pas représenter parfaitement Q*) est tellement dominante par rapport aux différences entre λ=0 et λ=1 que ces différences deviennent statistiquement indiscernables.

Comparaison de l'erreur quadratique moyenne entre λ=0 et λ=1 :

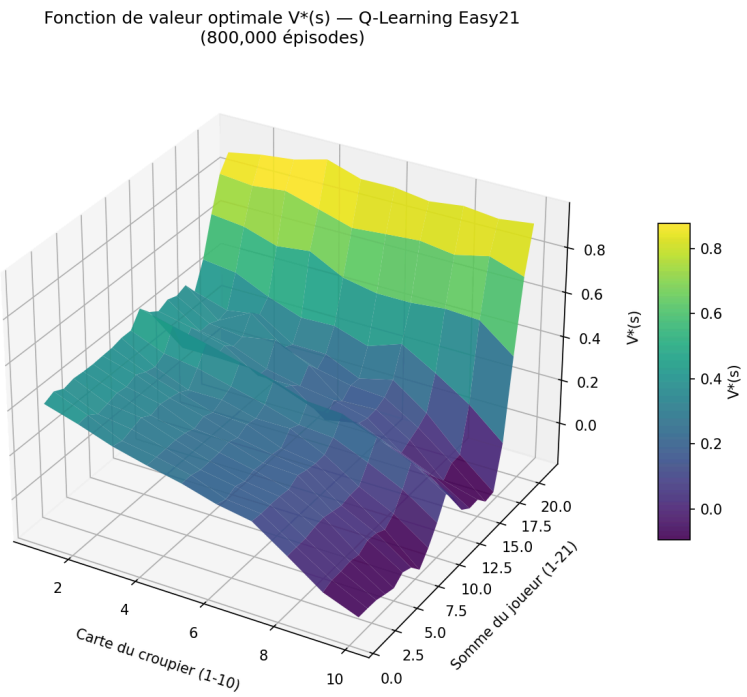


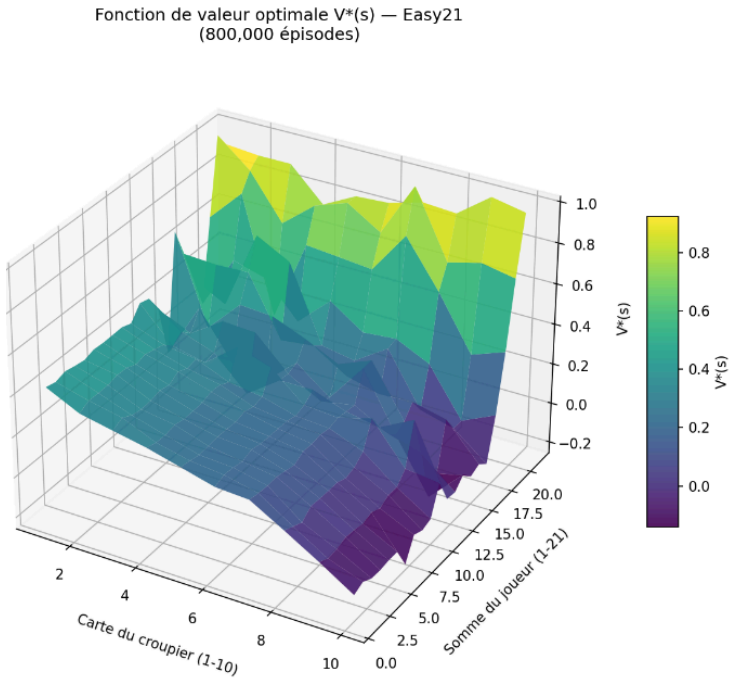
Sarsa avec approximation linéaire converge plus rapidement mais les courbes sont plus bruitées et l'écart-type plus élevé. Cela signifie qu'avec l'approximation linéaire, Sarsa généralise plus rapidement mais ensuite, l'algorithme n'arrive pas à se stabiliser à la solution optimale à cause du codage grossier de l'approximateur qui ne peut donc pas représenter parfaitement Q*.

Q-learning :



Ces résultats sont cohérents avec la théorie. Easy21 étant un processus de Markov, les méthodes TD à un pas (Q-Learning et Sarsa(λ=0)) exploitent mieux cette structure markovienne en bootstrappant, ce qui leur donne une variance faible et une convergence rapide et stable. Sarsa(λ=1), équivalent à Monte-Carlo, ignore cette structure et souffre de la forte variance des retours. Le fait que Q-Learning n'apporte aucun gain supplémentaire sur Sarsa(λ=0) suggère que la distinction on-policy/off-policy est peu importante dans Easy21, où l'espace d'états est petit et bien couvert.





La surface Q-Learning (gauche) est lisse et régulière. Les transitions entre états voisins sont douces, sans discontinuités abruptes. Cela reflète le comportement de Q-Learning qui bootstrappe à chaque pas : les valeurs de Q(s,a) se propagent progressivement aux états voisins via les mises à jour TD, créant naturellement un effet de lissage spatial.

La surface Monte-Carlo (droite) est beaucoup plus accidentée et irrégulière, avec des pics et des creux locaux prononcés, notamment dans la zone des sommes intermédiaires. Cela est dû à la haute variance de Monte-Carlo : chaque état est estimé indépendamment à partir des retours observés sur les épisodes qui l'ont visité, sans partager d'information avec les états voisins. Un état peu visité peut avoir une estimation très éloignée de ses voisins, créant ces irrégularités.

Les deux surfaces convergent vers la même vérité qualitative mais Monte-Carlo est plus bruitée car chaque état est estimé de façon isolée. Q-Learning produit une représentation plus cohérente spatialement grâce au bootstrapping, au prix d'un léger biais.

Discussion :

Q1 : Quels sont les avantages et les inconvénients du bootstrapping dans Easy21 ?

Le bootstrapping consiste à mettre à jour la fonction de valeur en utilisant une estimation existante de la valeur d'un état suivant, plutôt qu'en attendant le retour réel de fin d'épisode. Dans Sarsa(λ), cela correspond aux valeurs λ proches de 0.
Le premier avantage est la réduction de la variance. Le bootstrapping lisse les fluctuations en s'appuyant sur l'estimateur courant plutôt que sur un retour complet.
Le second avantage est la mise à jour en ligne et incrémentale. Le bootstrapping permet de mettre à jour Q(s,a) à chaque pas de temps sans attendre la fin de l'épisode, ce qui accélère la propagation de l'information dans les premiers états de la trajectoire.
Enfin, le bootstrapping offre une meilleure efficacité en données dans les premiers épisodes, car même avec peu d'expérience, l'estimateur courant fournit un signal d'apprentissage immédiat.
Le principal inconvénient est le biais introduit par l'estimateur. Si Q est mal initialisé ou peu précis, les mises à jour TD propagent et amplifient ces erreurs. Dans Easy21, où l'espace d'état est relativement grand, cet effet de propagation du biais peut ralentir la convergence.
De plus, avec un bootstrapping pur (λ = 0), la propagation de la valeur est lente : il faut de nombreux épisodes pour qu'une récompense reçue en fin de partie remonte jusqu'aux états initiaux, car chaque mise à jour ne porte que sur une transition à la fois.

Q2 : Le bootstrapping serait-il plus utile dans le Blackjack classique ou dans Easy21 ?

Le bootstrapping serait plus utile dans Easy21 que dans le Blackjack classique, car :
Dans le Blackjack classique, les épisodes sont courts et la dynamique est peu stochastique du point de vue du joueur : on tire simplement des cartes avec une valeur positive. Les retours Monte Carlo sont donc peu bruités et les épisodes se terminent rapidement. Le gain du bootstrapping y est limité.
Dans Easy21 en revanche, les épisodes peuvent être nettement plus longs car les cartes rouges peuvent faire diminuer la somme du joueur, forçant des épisodes plus longs. De plus, la stochasticité issue des cartes pouvant soustraire la somme, augmente la variance des retours Monte Carlo. Le bootstrapping est donc particulièrement bénéfique ici car il permet de réduire cette variance tout en permettant des mises à jour fréquentes.

Q3 : Quels sont les avantages et les inconvénients de l'approximation linéaire de la fonction de valeur par rapport à la représentation tabulaire dans le contexte d'Easy21 ?
La généralisation apportée par le codage grossier est-elle bénéfique ici ?

L'approximation de la fonction de valeur consiste à représenter Q(s,a) non plus par une table mais par une fonction paramétrique (par exemple linéaire avec des features binaires encodant des intervalles d'états).
Le premier avantage est la généralisation. Avec une table, chaque case (s,a) est apprise indépendamment. Avec un approximateur, les états similaires partagent leurs paramètres, ce qui permet de généraliser l'expérience acquise dans un état à des états voisins non encore visités.
Le second avantage est la compacité de la représentation. Un approximateur linéaire avec quelques dizaines de features suffit à capturer les grandes tendances de Q*, là où la table nécessite 420 paramètres distincts, chacun devant être visité suffisamment souvent.
Le principal inconvénient est le biais de représentation. Si les features choisies ne permettent pas d'exprimer la vraie forme de Q*, l'approximateur ne convergera pas vers Q* même avec un nombre infini d'épisodes.
De plus, la convergence n'est pas garantie lorsqu'on combine approximation de fonction et bootstrapping, contrairement au cas tabulaire. Des oscillations ou des divergences peuvent apparaître, surtout si le taux d'apprentissage n'est pas bien calibré.
La généralisation du codage grossier peut sembler utile, mais en pratique les intervalles grossiers introduisent un biais structurel important. Easy21 a un espace d'états plutôt petit donc la représentation tabulaire converge bien vers une solution. La généralisation du codage grossier serait vraiment bénéfique dans des espaces de grande dimension où visiter chaque état est impossible.

Q4 : Comment modifieriez-vous l'approximateur de fonction proposé pour obtenir de meilleurs résultats ?

Pour améliorer l'approximateur, on pourrait dans un premier temps affiner les intervalles actuels qui sont assez larges. Réduire leur taille ou en ajouter davantage permettrait une représentation plus fine de Q*, au prix d'un vecteur φ plus grand et d'un risque de moins de généralisation.
Une autre solution serait de passer à un réseau de neurones qui permettrait de capturer des interactions non-linéaires entre la carte du croupier, la somme du joueur et l'action, ce que l'approximation linéaire ne peut pas faire par construction.

Q5 : Expliquer la différence fondamentale entre Q-Learning (hors-politique) et Sarsa (en-politique). Dans le cas d'Easy21, quel algorithme converge le plus rapidement ? Cette observation est-elle cohérente avec la théorie ?

La différence se situe dans la règle de mise à jour de la fonction Q.
Sarsa adapte la politique courante, tandis que Q-Learning apprend directement la politique optimale.

En pratique dans Easy21, les deux méthodes convergent vers la même solution, mais Q-Learning a généralement une MSE finale plus faible après un même nombre d'épisodes, car il tire mieux parti de chaque transition.
Sarsa(λ=0) peut être compétitif ou supérieur car les traces d'éligibilité propagent les récompenses efficacement sur des épisodes courts comme Easy21.
Cette observation est cohérente avec la théorie car Q-Learning converge plus vite en termes de données (efficacité par épisode).
Mais Sarsa profite mieux de la structure temporelle des épisodes, ce qui peut compenser.

Q6 : En vous appuyant sur vos résultats des deux TP, quel algorithme (parmi Monte-Carlo, Sarsa(λ) avec représentation tabulaire, Sarsa(λ) avec approximation linéaire et Q-Learning) recommanderiez-vous pour jouer à Easy21 ?
Justifier en prenant en compte les critères de vitesse de convergence, qualité de la solution et complexité d'implémentation.

Q-Learning et Sarsa(λ=0) tabulaire convergent le plus vite en pratique.
Monte-Carlo est le plus lent car il attend la fin de l'épisode avant toute mise à jour, et la variance des retours est élevée.
Monte-Carlo avec suffisamment d'épisodes donne la meilleure estimation de Q* (pas de biais de bootstrapping). Q-learning tabulaire converge vers la même solution avec moins d'épisodes. L'approximation linéaire introduit un biais qui dégrade légèrement la politique finale et n'est pas vraiment bénéfique de le cas d'Easy21 qui a peu d'états.
Enfin, Monte-Carlo et les méthodes tabulaires (Sarsa, Q-Learning) sont plutôt simples à implémenter alors que l'approximation linéaire ajoute la gestion du vecteur de caractéristiques.
Ainsi, pour jouer à Easy21, je recommande donc Q-learning qui converge vers une bonne solution rapidement tout en ayant une implémentation relativement simple.